



SECURECORE

Information Security Management System

Security Project Documentation

Version 1.1 | 2025

Prepared by: John Carl D. Restauero
IT16L(8465)

1. Project Overview

System Description

SecureCore is a web-based Employee Information and Security Management System built with Laravel (PHP) and MySQL. It provides a centralized platform for managing employee records, roles, departments, and access permissions with a strong focus on data security and access control.

Purpose of the System

The system aims to:

- Provide secure employee data management with role-based access control.
- Protect user accounts through multi-layered authentication security.
- Prevent common cyber threats such as brute-force attacks, unauthorized access, and credential leakage.
- Ensure sensitive data such as contact numbers and email addresses are protected based on user roles.
- Maintain a complete audit trail of all authentication and administrative events.

Intended Users

The system is designed for three types of users:

- Admin — Full system access including user management, security logs, and all employee data unmasked.
- Manager — Department-scoped access with masked sensitive data (contact numbers and emails).
- Employee — Read-only access to their own profile with full visibility of their own data.

Platform and Technology Used

Technology Stack

- Programming Language: PHP 8.1.25
- Framework: Laravel 10.50.2
- Database: MySQL
- Frontend: Blade Templates, HTML5, CSS3, Vanilla JavaScript
- Authentication: Laravel Auth + reCAPTCHA v2
- Platform: Web (localhost / production server)
- Security Libraries: Enlightn, PHP Security Checker

2. Secure Coding Practices

Environment Variables

All sensitive credentials and configuration values are stored in the .env file and never hardcoded into application source code. This includes database credentials, API keys, reCAPTCHA keys, and application secrets.

Credentials Managed via .env

- DB_PASSWORD — Database password
- APP_KEY — Laravel application encryption key
- RECAPTCHA_SITE_KEY / RECAPTCHA_SECRET_KEY — Google reCAPTCHA v2 keys
- SESSION_SECURE_COOKIE — Toggled per environment
- APP_ENV / APP_DEBUG — Environment and debug mode flags

CSRF Protection

All POST, PUT, PATCH, and DELETE requests include a @csrf Blade directive which Laravel validates automatically via the VerifyCsrfToken middleware, preventing cross-site request forgery attacks.

XSS Prevention

All output rendered in Blade templates uses the {{ }} syntax which automatically escapes HTML entities. The {!! !!} raw output syntax is never used with user-supplied data, preventing cross-site scripting.

SQL Injection Prevention

All database interactions use Laravel's Eloquent ORM and the Query Builder which use PDO prepared statements with parameter binding. No raw SQL is constructed from user input.

Debug Mode

APP_DEBUG is set to true during local development only. When deploying to production, it is set to false to prevent stack traces and sensitive system details from being exposed to end users.

3. Authentication and Authorization

Login Process

The authentication flow follows these steps:

1. User submits email, password, and reCAPTCHA v2 token via the login form.
2. reCAPTCHA v2 checkbox is verified with Google's API — uncompleted or failed verifications are rejected.
3. The system checks whether the account is currently locked due to prior failed attempts.
4. Laravel's `Auth::attempt()` verifies credentials against the hashed password in the database.
5. On success: session is regenerated, failed attempt counter is reset, and the event is logged.
6. On failure: failed attempt counter is incremented and the remaining attempts are shown.
7. After 5 consecutive failures: the account is locked for 5 minutes automatically.

Password Hashing

Passwords are never stored in plain text. Laravel's User model uses the hashed cast which applies bcrypt hashing automatically on every password assignment. The bcrypt algorithm includes salting to prevent rainbow table attacks.

Password Security Policy

- Minimum 12 characters, maximum 16 characters
- Must include at least one uppercase letter (A-Z)
- Must include at least one lowercase letter (a-z)
- Must include at least one number (0-9)
- Must include at least one symbol (!@#\$\$%^&*...)
- Confirm password field enforced on creation
- Real-time strength meter and policy checklist shown during creation

Account Lockout Policy

The system implements a database-backed account lockout mechanism to prevent brute-force attacks:

- 5 consecutive failed login attempts triggers an automatic lockout.
- Locked accounts cannot authenticate for 5 minutes.
- A live countdown timer is displayed on the login page during lockout.
- All inputs and the submit button are disabled during the lockout period.
- The lockout automatically lifts after the timer expires — no manual intervention needed.
- Successful login resets the failed attempt counter to zero.

Session Security

Sessions are hardened with the following configuration in `config/session.php`:

- `http_only`: true — cookies are inaccessible to JavaScript.
- `same_site`: strict — cookies are not sent on cross-site requests.

- `secure: true` (production) — cookies are only transmitted over HTTPS.
- `encrypt: true` — all session data is encrypted at rest.
- `lifetime: 30 minutes` — sessions expire after 30 minutes of inactivity.

User Roles

The system implements three distinct roles stored in the `role` column of the `users` table:

- `admin` — Full system access, unmasked data, all CRUD operations, security logs.
- `manager` — Department-scoped read/edit access, masked sensitive data.
- `employee` — Read-only access to own profile with unmasked personal data.

4. Data Encryption

Password Encryption

All user passwords are hashed using bcrypt via Laravel's built-in hashed cast on the User model. Bcrypt is a one-way hashing algorithm that includes automatic salting, making it resistant to rainbow table and brute-force attacks.

Session Encryption

Laravel session data is encrypted using the application's APP_KEY (a 256-bit AES key) when the encrypt option is enabled in config/session.php. This ensures session contents cannot be read even if the session storage is compromised.

Sensitive Field Protection (Data Masking)

While fields such as contact_number and address are stored as plain text in the database, access is controlled and masked at the application layer based on the authenticated user's role:

- Admin — sees all data unmasked.
- Manager — sees email as j*****@gmail.com and contact as 0917***4567.
- Employee — sees only their own data, fully unmasked on their private dashboard.

Masking is performed server-side in the controller before data is passed to the view, ensuring masked values are never sent to the browser in plain text for unauthorized roles.

Transport Encryption (HTTPS)

The AppServiceProvider is configured to enforce HTTPS in production environments. When APP_ENV is set to production, all URLs are forced to use the HTTPS scheme via URL::forceScheme('https'), ensuring all data in transit is encrypted via TLS.

5. Input Validation and Sanitization

Validated Inputs

All user-submitted data is validated server-side using Laravel's built-in validation system before any processing or database operation occurs. The following inputs are validated:

- Login: email (required, valid email format), password (required).
- Employee Create/Edit: name, email (unique), password (complexity rules), role, department_id, contact_number, address.
- reCAPTCHA: g-recaptcha-response (required, checkbox must be completed and verified by Google).

Password Complexity Validation

Password validation enforces the security policy using Laravel's regex rule on the backend, in addition to real-time client-side feedback:

Backend Validation Rule

- required | string | confirmed | min:12 | max:16
- regex: must contain uppercase, lowercase, number, and symbol
- Custom error messages returned for each failed rule

XSS and Output Sanitization

All data rendered in Blade views uses the `{{ }}` double-brace syntax which automatically applies `htmlspecialchars()` to the output, preventing cross-site scripting attacks. User input is never rendered with `{!! !!}` raw output.

Client-Side Validation

Client-side validation provides immediate feedback to users but is never relied upon as the sole validation layer. All validation is duplicated server-side. Client-side features include:

- Real-time password strength meter with 5-level scoring.
- Live policy checklist with pass/fail icons for each password rule.
- Real-time confirm password match indicator.
- Form submission blocked in JavaScript if policy is not met.

6. Error Handling and Logging

Secure Error Handling

The application is configured with `APP_DEBUG=false` in production, which prevents detailed error messages, stack traces, and system paths from being exposed to end users. Laravel's default exception handler returns generic error pages instead.

Authentication errors return generic messages to avoid leaking information about account existence. For example, whether an email is not registered or the password is wrong, the user always sees the same 'Invalid credentials' message.

Security Logging

All security-relevant events are written to a dedicated security log channel (`storage/logs/security.log`) using Laravel's Log facade. The following events are logged:

Logged Security Events

- Successful login — user_id, email, IP address, user agent
- Failed login attempt — email, IP address, user agent, attempt count
- Login attempted on locked account — email, IP address, locked_until timestamp
- Account locked — triggered after 5th consecutive failure
- User logout — user_id, IP address
- Employee created — admin_id, new employee_id, email
- Employee updated — updated_by, employee_id, email
- Employee deleted — admin_id, employee_id, email
- reCAPTCHA v2 failure — email, failed verification response, IP address

Log Viewer

Admins can view security logs through the Admin Panel's Security Logs page. The log viewer provides:

- Newest-first ordering of log entries.
- Type filtering (All Events, Successful Logins, Failed Attempts, Logouts).
- Date range filtering with from/to date pickers.
- Full-text search across all log fields.
- Paginated display of 50 entries per page.
- Summary statistics chips (total events, successes, failures, logouts).

7. Access Control

Middleware Protection

All authenticated routes are protected by Laravel's auth middleware, which redirects unauthenticated users to the login page. The `EmployeeController` constructor applies this middleware to all its methods.

Policy-Based Authorization

Fine-grained access control is implemented using Laravel Policies. Each controller action calls `$this->authorize()` before executing, which checks the authenticated user's role against the defined policy rules:

- `viewAny` — admin and manager only.
- `create` — admin only.
- `update` — admin (any employee), manager (department-scoped only).
- `delete` — admin only.

Role-Based Access Control (RBAC) Matrix

System Feature / Resource	Guest	Employee	Manager	Admin
View Login Page	Allowed	Allowed	Allowed	Allowed
Login	Allowed	Allowed	Allowed	Allowed
Employee Dashboard	Denied	Allowed	Denied	Denied
Admin Dashboard	Denied	Denied	Denied	Allowed
View Own Profile	Denied	Allowed	Denied	Denied
View Department Users	Denied	Denied	Allowed	Allowed
View All Users	Denied	Denied	Denied	Allowed
View Masked Contact/Email	Denied	Denied	Allowed	Denied
View Full Contact/Email	Denied	Denied	Denied	Allowed
Add Employee	Denied	Denied	Denied	Allowed
Edit Employee	Denied	Denied	Allowed	Allowed
Delete Employee	Denied	Denied	Denied	Allowed
View Security Logs	Denied	Denied	Denied	Allowed
Change Role/Department	Denied	Denied	Denied	Allowed

8. Code Auditing Tools

Enlightn — Laravel Security Auditor

Enlightn is a Laravel-specific static analysis tool that scans the application for security vulnerabilities, performance issues, and reliability problems.

Enlightn Audit Results

- Total Checks: 67
- Passed: 50 (75%)
- Failed: 12 (18%) — primarily debug-mode and HTTP-only flags expected in dev
- Not Applicable: 5 (7%)
- Security Passed: 15 of 21 (71%)
- Remaining failures are environment-specific and will pass in production

PHP Security Checker

The PHP Security Checker scans the composer.lock file against the Security Advisories Database to identify any third-party dependencies with known CVEs. The command `php artisan security:check` is run as part of the development workflow.

Security Headers Middleware

The `bepsvpt/secure-headers` package is installed and configured to automatically add HTTP security headers to all responses including Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy.

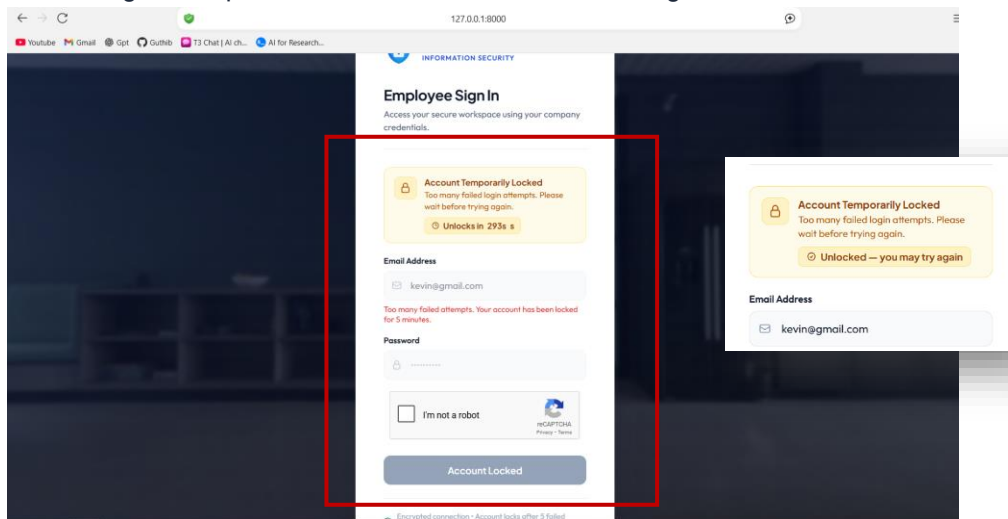
Manual Code Review Checklist

- No hardcoded credentials in source files.
- No use of `{!! !!}` with user-supplied data in Blade views.
- No raw `DB::statement()` with unbound user input.
- All forms include `@csrf` directive.
- All controller actions call `$this->authorize()` before executing.

9. Testing

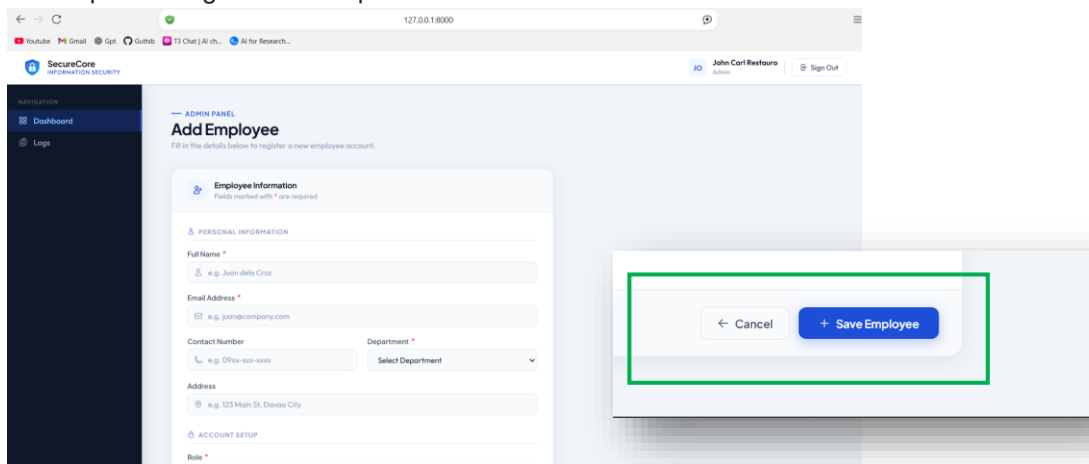
Authentication Testing

- Valid credentials with passing reCAPTCHA — login succeeds, session created, redirected to dashboard.
- Invalid password — error shown with remaining attempts count.
- 5 consecutive failures — account locked, countdown timer displayed, form disabled.
- Login attempt during lockout — blocked with locked message and remaining time.
- After lockout expires — form re-enables, countdown shows 'Unlocked'.
- Correct login after previous failures — counter resets, login succeeds.



Authorization Testing

- Employee accessing /employees route — redirected, access denied.
- Manager accessing another department's employee edit — policy blocks the action.
- Manager viewing logs — access denied by middleware.
- Admin performing all CRUD operations — all succeed.



Input Validation Testing

- Password shorter than 12 characters — form blocked client-side and server-side.
- Password without symbol — strength meter shows incomplete, server rejects.
- Mismatched confirm password — match indicator shows red, submit blocked.
- Duplicate email on create — server returns unique validation error.
- SQL injection attempt in email field — Eloquent parameterized query neutralizes it.

The screenshot shows a password creation form. The 'Password' field is highlighted with a red border. Below the field, a strength meter is labeled 'Weak'. A list of requirements is shown with checkmarks or crosses: '12-16 characters' (X), 'Lowercase letter (a-z)' (checkmark), 'Symbol (!@#\$%^&*...)' (X), 'Uppercase letter (A-Z)' (X), and 'Number (0-9)' (checkmark). Below the password field is a 'Confirm Password' field with the placeholder text 'Re-enter password'. At the bottom of the form are two buttons: 'Cancel' and '+ Save Employee'.

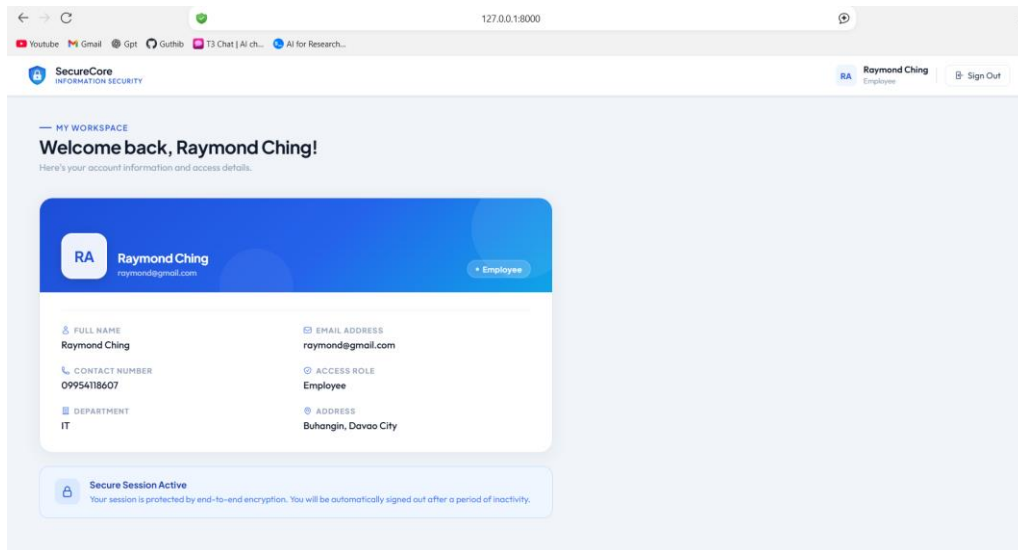
Data Masking Testing

- Manager login — contact numbers show as 0917***4567, emails show as j*****@domain.com.
- Admin login — all data shown unmasked.
- Employee dashboard — own contact and address shown in full.

The screenshot shows an 'Employee Directory' table. The table has columns for #, EMPLOYEE, EMAIL, CONTACT, ROLE, DEPARTMENT, and ACTIONS. The data is as follows:

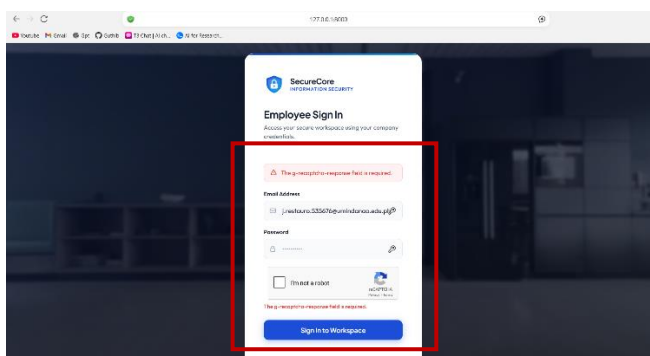
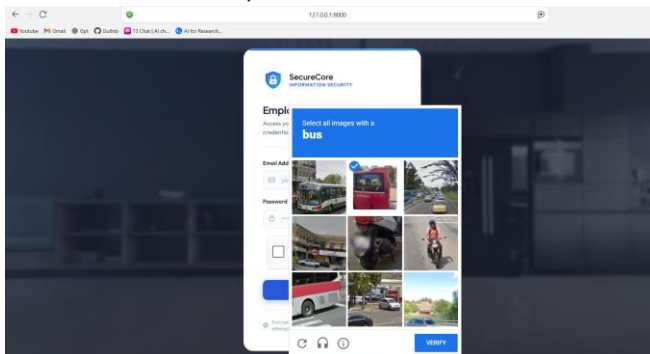
#	EMPLOYEE	EMAIL	CONTACT	ROLE	DEPARTMENT	ACTIONS
#535677	EM Emmanuel Felipe	e****@gmail.com	0995***8607	Manager	IT	Edit Delete
#535678	RA Raymond Ching	j*****@gmail.com	0995***8607	Employee	IT	Edit Delete
#535681	KE Kevin Merioles	k****@gmail.com	0995***2607	Employee	IT	Edit Delete

The 'EMAIL' and 'CONTACT' columns for the first two rows are highlighted with a red border, indicating data masking.



reCAPTCHA v2 Testing

- Valid checkbox completion — token generated, verification passes, login proceeds.
- Skipping the checkbox — form blocked, validation fails with 'CAPTCHA verification failed' error message.
- After multiple failed attempts — Google presents image challenges (buses, traffic lights) as risk score increases. This is expected v2 behavior.



10. Security Policies

Policy	Details
Password Policy	Minimum 12, maximum 16 characters. Must contain uppercase, lowercase, number, and symbol. Enforced at creation via regex validation on both client and server. Passwords are hashed with bcrypt and never stored in plain text.
Account Lockout Policy	Accounts are locked for 5 minutes after 5 consecutive failed login attempts. Lockout state is stored in the database (locked_until column). A live countdown is shown. The form is disabled during lockout and re-enables automatically.
Session Policy	Sessions expire after 30 minutes of inactivity. Session cookies are HTTP-only, SameSite=Strict, and Secure (production). Session data is encrypted. Sessions are regenerated on login to prevent fixation attacks.
Data Access Policy	Sensitive fields (contact number, email) are masked for manager-role users. Only admins see unmasked data. Masking is enforced server-side. Employees may only view their own profile data.
Authentication Policy	reCAPTCHA v2 checkbox is required on every login attempt. Failed or missing CAPTCHA responses are rejected by Google's verification API. Login events (success, failure, lockout) are logged with IP address, timestamp, and user agent.
Logging and Monitoring Policy	All authentication events are written to the security log channel. Logs are accessible only to admin users through the Admin Panel. Logs include timestamps, IP addresses, user agents, and event-specific context.
Access Control Policy	All routes require authentication. All controller actions enforce role-based authorization via Laravel Policies. Role and department assignment is restricted to admins only.
HTTPS Policy	All production traffic is forced to HTTPS via URL::forceScheme in AppServiceProvider. Secure cookies are enabled in production. HTTP connections are not accepted for authenticated sessions.
Debug and Error Policy	APP_DEBUG is false in production. Stack traces and system paths are never exposed to end users. Authentication errors use generic messages to avoid leaking account existence.
Backup and Recovery Policy	Database backups should be scheduled weekly and stored in a secure off-site location. The .env file and APP_KEY must be backed up securely as they are required for session and encryption recovery.

11. Incident Response Plan

Detection

Security incidents are detected through multiple channels:

- Security log monitoring — repeated failed login attempts, unusual IP addresses, or access violations are visible in the Admin Panel's Security Logs page.
- Account lockout alerts — when an account is locked, the event is logged with full context including IP address and timestamp.
- Enlightn scheduled audits — weekly automated scans flag newly introduced vulnerabilities.
- PHP Security Checker — new CVEs in dependencies are detected via composer.lock scanning.

Reporting

Upon detection of a security incident:

8. The admin reviews the security log for the affected account or IP address.
9. The incident is documented including timestamp, affected user, IP address, and nature of the attempt.
10. If a breach is confirmed, the system administrator and relevant stakeholders are notified immediately.
11. The incident is escalated to the appropriate authority if it involves sensitive data exposure.

Containment

Immediate containment measures include:

- Manually extending the account lockout by updating the locked_until field in the database if the automated 5-minute lockout is insufficient.
- Disabling the affected user account by changing their role or removing their record if necessary.
- Blocking the offending IP address at the server or firewall level.
- Revoking active sessions by invalidating session tokens in the session storage.
- Rotating the APP_KEY if a session compromise is suspected (note: this invalidates all active sessions).

Recovery

System recovery steps following a confirmed incident:

12. Verify the integrity of the affected user accounts and reset passwords if necessary.
13. Review all security logs for the affected time window to determine the full scope.
14. Patch any identified vulnerability and re-run Enlightn to confirm resolution.
15. Restore from the most recent clean backup if data integrity is compromised.
16. Notify affected users of any data exposure and required password changes.
17. Conduct a post-incident review and update security policies if necessary.
18. Document the full incident timeline, response actions, and lessons learned.